FULL STACK PROJECT

Student Feedback System

A web-based system to collect and view student feedback using React.js and Express.js



1. Aim

OBJECTIVE

To design and develop a simple web-based Student Feedback System that allows students to submit feedback about a subject and lets everyone view all submitted feedback in a table. React.js builds the frontend and Express.js provides the backend.

1

Submit

Student enters name, roll number, subject, rating and written feedback.

2

View

All submitted feedback is fetched from the server and shown in a table.

3

Store

Feedback is kept temporarily in an array in Express. No database needed.

2. Project Explanation

1

Enter details

Name and roll number.

2

Choose & rate

Select subject and give a rating.

3

Submit

Write feedback and click Submit.

4

View table

Feedback appears in the table.

Key Features

- Student name and roll number input
- Subject selection and 1-5 rating
- Feedback message with validation
- Submit and view all feedback in a table

Technologies Used

HTML: Page structure

CSS: Design

JavaScript: Functionality

React.js: Frontend

Express.js / Node.js: Backend and server

3. Procedure

1. Open VS Code and a Git Bash terminal
2. Create the project folder Student-Feedback
3. Create the server folder and run npm init -y
4. Install Express and CORS: npm install express cors
5. Create server.js with GET and POST routes
6. Create the React app using Vite
7. Run npm install inside the client folder
8. Write the form and table in App.jsx
9. Add the styling in App.css
10. Terminal 1: cd server, then node server.js
11. Terminal 2: cd client, then npm run dev
12. Open http://localhost:5173 in Chrome
13. Submit feedback and check the table

Backend runs on port 5000, frontend on port 5173

4. Source Code: Server Setup

Express server (server.js) with middleware, in-memory array and GET route

```
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

let feedbacks = [];

// Get all feedback
app.get("/feedback", (req, res) => {
    res.json(feedbacks);
});

app.listen(5000, () => {
    console.log("Server running on port 5000");
});
```

4. Source Code: POST Route

Validate the data and add new feedback to the array

```
// Add feedback
app.post("/feedback", (req, res) => {
    const { name, roll, subject, rating, message } = req.body;

    if (!name || !roll || !subject || !rating || !message) {
        return res.status(400).json({ message: "Fill all fields" });
    }

    const feedback = {
        id: Date.now(),
        name, roll, subject, rating, message
    };

    feedbacks.push(feedback);
    res.json({ message: "Feedback submitted successfully!" });
});
```

4. Source Code: Imports and State

React hooks keep each form field and the feedback list (App.jsx)

```
import React, { useState, useEffect } from "react";
import "./App.css";

function App() {
    const [name, setName] = useState("");
    const [roll, setRoll] = useState("");
    const [subject, setSubject] = useState("");
    const [rating, setRating] = useState("");
    const [message, setMessage] = useState("");
    const [feedbacks, setFeedbacks] = useState([]);
```

4. Source Code: Fetching Feedback

Load all feedback from the server when the page opens

```
const getFeedback = async () => {
    const res = await fetch("http://localhost:5000/feedback");
    const data = await res.json();
    setFeedbacks(data);
};

useEffect(() => {
    getFeedback();
}, []);
```

4. Source Code: Submitting Feedback

Send the form data to the server with a POST request

```
const submitFeedback = async (e) => {
    e.preventDefault();
    const feedback = { name, roll, subject, rating, message };

    const res = await fetch("http://localhost:5000/feedback", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(feedback)
    });

    const data = await res.json();
    alert(data.message);

    if (res.ok) {
        setName(""); setRoll(""); setSubject("");
        setRating(""); setMessage("");
        getFeedback();
    }
};
```

4. Source Code: Feedback Form

Controlled inputs, subject and rating dropdowns, and the submit button

```
<form onSubmit={submitFeedback}>
    <input placeholder="Student Name" value={name}
        onChange={(e) => setName(e.target.value)} required />
    <input placeholder="Roll Number" value={roll}
        onChange={(e) => setRoll(e.target.value)} required />

    <select value={subject} onChange={(e) => setSubject(e.target.value)} required>
        <option value="">Select Subject</option>
        <option>React JS</option>  <option>Express JS</option>
    </select>

    <select value={rating} onChange={(e) => setRating(e.target.value)} required>
        <option value="">Give Rating</option>
        <option value="5">Excellent - 5</option>  ...
    </select>

    <textarea placeholder="Write your feedback" value={message} required />
    <button type="submit">Submit Feedback</button>
</form>
```

4. Source Code: Feedback Table

Render every submitted feedback as a table row with feedbacks.map()

```
<table>
    <thead>
        <tr>
            <th>Name</th> <th>Roll No</th> <th>Subject</th>
            <th>Rating</th> <th>Feedback</th>
        </tr>
    </thead>
    <tbody>
        {feedbacks.map((f) => (
            <tr key={f.id}>
                <td>{f.name}</td>  <td>{f.roll}</td>
                <td>{f.subject}</td>  <td>{f.rating}/5</td>
                <td>{f.message}</td>
            </tr>
        ))}
    </tbody>
</table>
```

4. Source Code: CSS Design

Key styles from App.css for the page, form and table

```
body { margin: 0; font-family: Arial, sans-serif; background: #eef2f7; }

.container { width: 85%; margin: 30px auto; text-align: center; }

form { background: white; padding: 25px; width: 350px; margin: 20px auto;
       border-radius: 10px; box-shadow: 0 2px 10px #ccc; }

input, select, textarea { width: 95%; padding: 12px; margin: 10px 0;
       border: 1px solid #ccc; border-radius: 5px; }

button { background: #263b80; color: white; padding: 12px; width: 100%;
       border: none; border-radius: 5px; cursor: pointer; }

table { width: 100%; background: white; border-collapse: collapse; }
th { background: #263b80; color: white; }
```

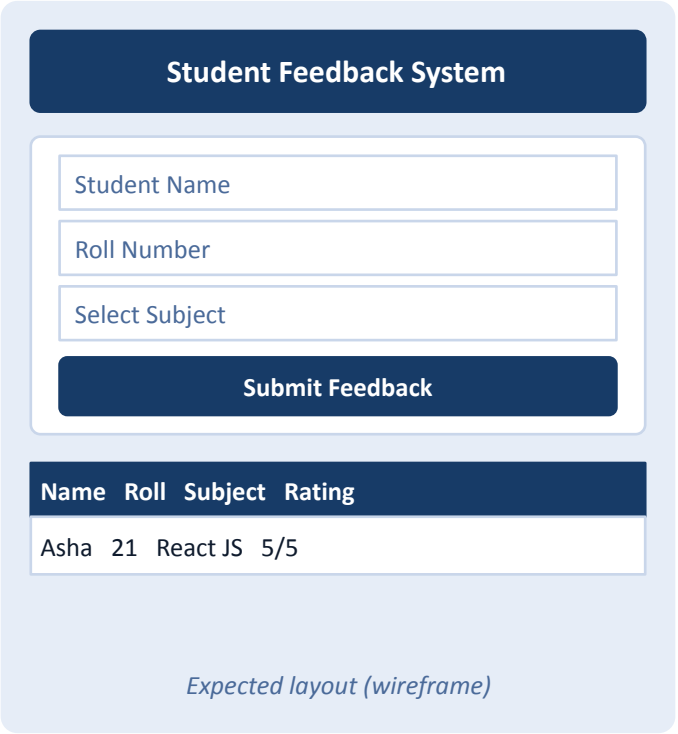
5. Concepts Used in the Project

Concept	Used in project	Concept	Used in project
Express Routing	GET and POST	**Props and State**	useState
HTTP Methods	GET and POST	**Events**	onChange and onSubmit
Middleware	cors and express.json	**Hooks**	useState and useEffect
Form Data	React form	**Rendering Lists**	feedbacks.map()
JSX	App.jsx	**Storage**	In-memory array in Express
Components	App component		

Note: this project does not use MongoDB. Restarting the Express server clears all feedback.

6. Expected Output

1. The page opens with the Student Feedback System heading and the form
2. Student enters name, roll number, subject, rating and feedback
3. Clicking Submit shows the alert: Feedback submitted successfully!
4. The form clears and the new entry appears in the Submitted Feedback table
5. Restarting the Express server clears the stored feedback



Expected layout (wireframe)

7. Output

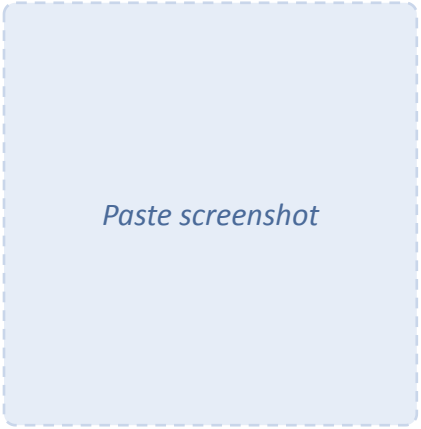


Fig 1. Empty form

Student Feedback System page with the form.

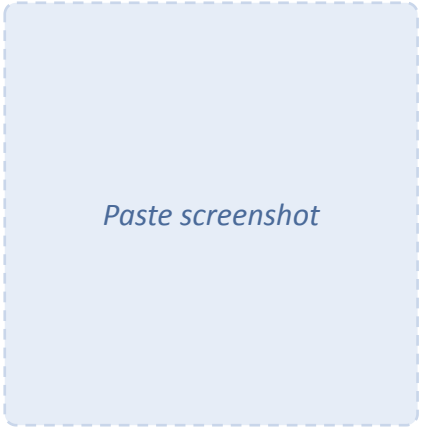


Fig 2. Filled form

Student fills all fields before submitting.

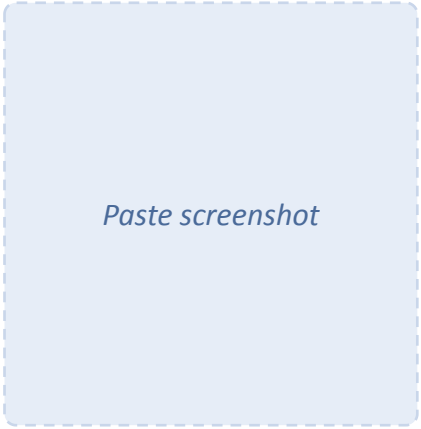


Fig 3. Success alert

Server confirms feedback was submitted.

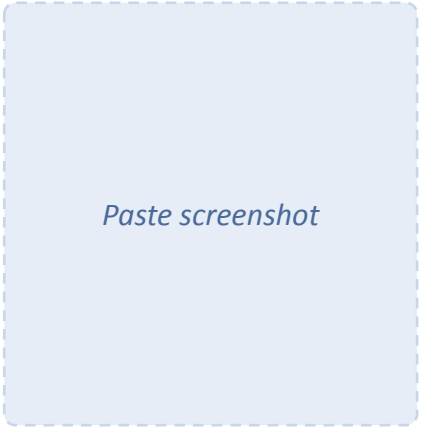


Fig 4. Feedback table

Submitted feedback shown in the table.

✓ **Feedback submitted by a student is stored by the Express server and displayed in the table.**

8. Result

- ✓ Students submit name, roll number, subject, rating and feedback
- ✓ Express server validates and stores each feedback
- ✓ React displays all feedback in a table
- ✓ Frontend and backend communicate using GET and POST



CONCLUSION

Thus, the Student Feedback System was successfully developed using React.js and Express.js. Feedback is stored temporarily in an array, so it is cleared when the server restarts.